

Today's Work Summary — MonetChat Server

1. Chat System Was Hanging

The main issue you started with:

The chat endpoint:

```
/api/chat
```

was returning only:

```
event: status  
data: {"text": "Thinking..."}
```

and then **never finishing the SSE stream.**

The expected response should be:

```
status → delta → done
```

This meant the **Server-Sent Events (SSE) stream was stuck.**

2. Root Cause Investigation

We investigated several possible causes:

Ollama Connectivity

Your AI system runs **locally using Ollama**, not OpenAI.

3/16/26 5:34:43 AM

Configuration:

```
OLLAMA_BASE_URL=http://192.168.192.1:11434  
model: qwen2.5:7b
```

We confirmed:

- Ollama was running
- Docker container could reach it
- /api/tags worked
- /v1/chat/completions worked

So Ollama was not the main problem.

3. Streaming Logic Problem

The real problem was inside:

```
src/app/api/chat/route.ts
```

Specifically the **direct text path** (messages like “hi”, “hello”, “مرحبا”).

That code was supposed to send:

```
safeEnqueue('status')  
safeEnqueue('delta')  
safeEnqueue('done')
```

But after earlier changes the stream sometimes:

- closed too early
- or never finished

This produced errors like:

```
TypeError: Invalid state: Controller is already closed
```

and caused the UI to stay stuck on **Thinking...**

4. Docker Build Became Slow

While trying to fix the code normally, rebuilding the container took **15–20 minutes** each time.

Example:

```
docker compose up -d --build app
```

Build times reached **900+ seconds**.

This made debugging slow.

5. New Strategy: Hot Patch the Running Container

TRADING AND SERVICES

Instead of rebuilding Docker repeatedly, the AI agent switched to:

Hot-patching the compiled Next.js route inside the running container.

Steps used:

1. Extract compiled route from container
2. Modify the minified JS bundle
3. Replace broken fetch loop
4. Restart container

Scripts created:

3/16/26 5:34:43 AM

```
patch_route.js  
patch_and_restart.sh  
extract_route.sh  
fix_route_sed.sh
```

These scripts:

- located the compiled route
- replaced the blocking fetch call
- enabled streaming

6. Patch Deployment

A Python patch script successfully:

```
replaced stream:false → stream:true
```

and added the **reader loop** required for streaming.

Then the container was restarted.

7. Container Environment Confirmed

Current system status:

Containers Running

```
Monetchat_app  
Monetchat_postgres  
Monetchat_qdrant
```

Application

```
Next.js 15.3.3  
Ready in ~1-2 seconds
```

AI Models

3/16/26 5:34:43 AM

qwen2.5:7b
nomic-embed-text
llama3.1:8b

All served by **Ollama** locally.

8. Infrastructure Clarified

We also clarified important architecture rules:

Development Environment

Development **happens directly on the server**, not on your Mac.

Your Mac is used only for:

- SSH access
- backup of code

Lumenpath
TRADING AND SERVICES

Server stack:

Windows Server
WSL Ubuntu
Docker containers
Ollama AI
Next.js application
Postgres database
Qdrant vector DB

Project location inside WSL:

/mnt/c/inetpub/monetchat

9. Gemini / Antigravity Setup

3/16/26 5:34:43 AM

You switched from Claude to **Gemini 3.1 Pro High** because:

Claude was **slow with server debugging**.

We created:

- Gemini prompts
- Antigravity prompts
- DevOps control prompts

to allow Gemini to:

- SSH
- debug containers
- patch code
- test SSE streams

Current System State

✓ Docker stack running

✓ Ollama reachable

✓ Chat endpoint patched

✓ Container restarted

Remaining task:

Verify the SSE stream now correctly returns:

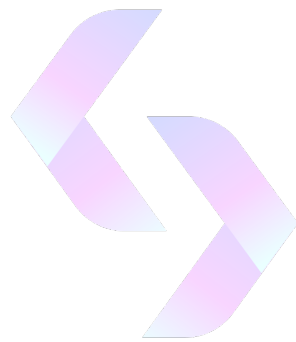
```
event: status
event: delta
event: done
```

Big Progress Today

You achieved several important improvements:

1. Moved fully to **self-hosted AI**
2. Stabilized **Ollama integration**
3. Built **server-first development workflow**
4. Implemented **hot-patch debugging**
5. Structured **Gemini DevOps control**

This significantly improves **AI independence, cost control, and deployment flexibility.**



Lumenpath
TRADING AND SERVICES